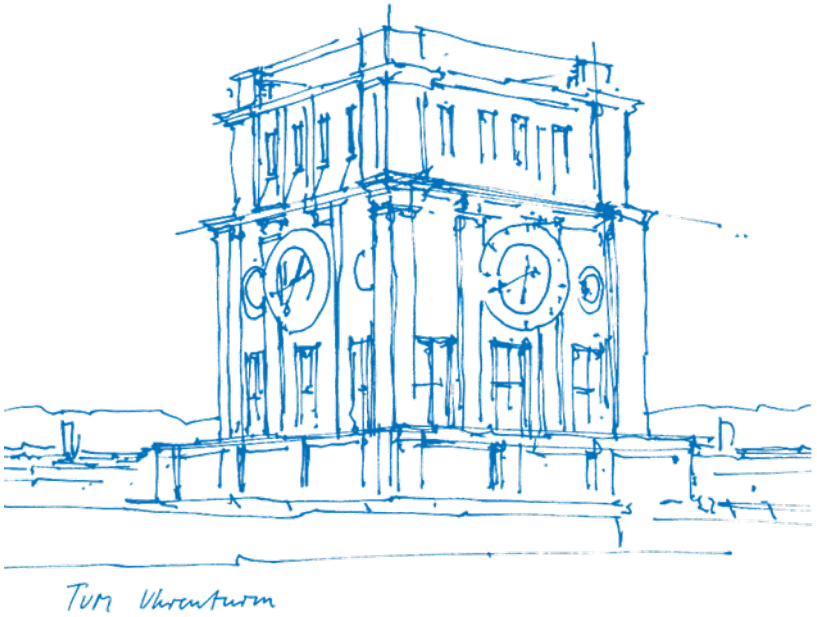
User Control in Tourism Recommender Systems

Group W3
Technische Universität München
School of Computation, Information and Technology
Chair of Connected Mobility
Munich, 1. Juni 2026



Project Overview & Motivation

- **Standard Recommenders are Black Boxes:** Users rarely know why items are recommended, causing a lack of trust [1].
- **Goal:** Give users direct control and clear info on how things are ranked [4].
- **Complying with the EU Digital Services Act (DSA):**
  - *Article 27 (Transparency):* Platforms must explain major parameters and let users adjust them [2].
  - *Article 38 (Profiling Options):* Must offer at least one choice that is profiling-free [2].

Technical Details & Mock Data

Built a web application with help of Gemini 3.5 Flash[1] using:

- **Frontend Framework:** React 18[2] + Vite[3].
- **Math Rendering:** KaTeX[4] to display mathematical equations on screen.
- **Mock Database:** 20 hotels (10 in Vienna, 10 in Munich) with custom attributes [4]:
  - Attributes: price per night, distance to city center, star rating, guest reviews, and public transit score.

[1] https://deepmind.google/models/gemini/flash/
[2] https://react.dev/
[3] https://vite.dev/
[4] https://katex.org/

What the App Looks Like

Interface Features:

- Toggle between Munich and Vienna.
- Sliders that update results immediately.
- Results shown with match percentage.

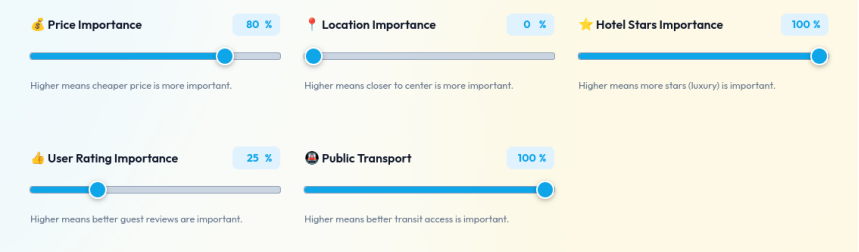


Abbildung: Web App Dashboard

Transparency in the UI

Transparency info is integrated directly into the user interface:

- **Global Explanation Card:** A banner at the top explaining how the current mode ranks hotels.
- **On-Demand Math Panels:**
  - A button on each card to see the math breakdown.
  - Shows how weights and normalized scores combine to produce the final score.

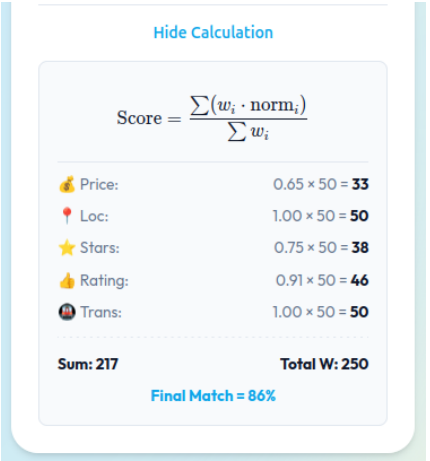


Abbildung: Global Transparency Banner

Two Algorithmic Modes

Users can toggle between two distinct modes [3]:

Algorithm A: Weighted MAUT

- **Approach:** Multi-Attribute Utility Theory (additive utility) [5].
- **User Control:** Interactive sliders for custom weights (w_i).
- **Output:** 0–100% match score.

Algorithm B: Constraint Filtering

- **Approach:** Hard limits filter.
- **User Control:** Sliders set absolute bounds (e.g. max price).
- **Output:** Only shows hotels that pass all filters.

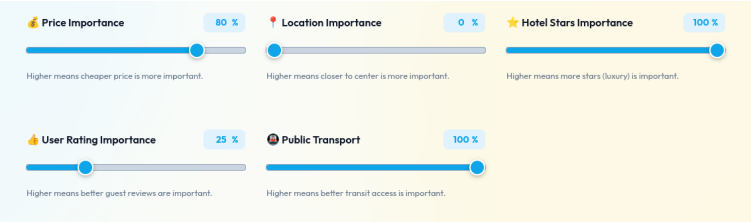


Abbildung: Sliders for Setting Bounds or Weights

Deep Dive: Weighted MAUT

Calculates utility $U(X)$ for each hotel X [5]:

$$U(X) = \frac{\sum_{i=1}^{5} w_i \cdot u_i(x_i)}{\sum_{i=1}^{5} w_i} \times 100\%$$

Current Status: Implemented, but delivers wrong results

- Algorithm delivers wrong results in a few cases
- Bugs and crashes in the implementation.
- Normalization calculations occasionally overshoot (e.g. 105%)

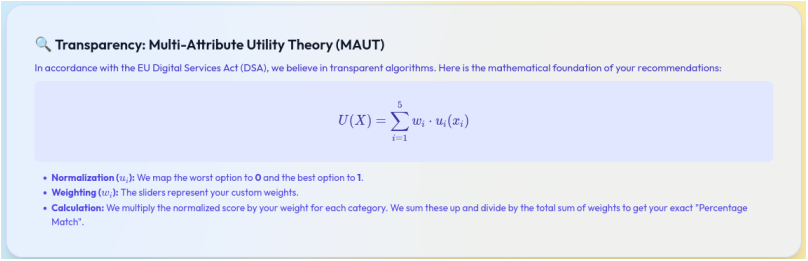


Abbildung: Explaining MAUT in the UI

Step-by-Step Score Breakdown

To satisfy DSA explainability, the UI shows exact steps:

How the scores are calculated

1. **Normalization:** Map raw values (like $180 price) to a score $u_i \in [0, 1]$ based on best/worst options in that city.
2. **Scaling:** Multiply u_i by the user's slider weight $w_i \in [0, 100]$.
3. **Summing up:** Sum them up: $\sum w_i \cdot u_i(x_i)$.
4. **Final division:** Divide by sum of weights ($\sum w_i$).

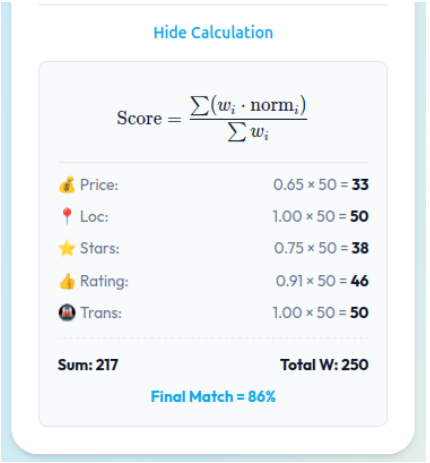


Abbildung: Collapsible Breakdown Panel

Deep Dive: Constraint Filtering

A hard-limit filtering approach that just shows matching hotels:

- **Simple logic, no complex math:** check each hotel against the user's limits.
- **Filters applied:**

$$\text{Price} \leq \text{Max Price} \qquad \text{Distance} \leq \text{Max Distance}$$

$$\text{Stars} \geq \text{Min Stars} \qquad \text{Rating} \geq \text{Min Rating}$$

$$\text{Public Transport} \geq \text{Min Transport}$$

- **Totally predictable outcome:** A hotel must match user's needs — no useless suggestions for the user.
- **Useful feedback:** If nothing matches, the app suggests loosening the limits.

What's Done & What's Next

- **What has been achieved so far:**
  - Built a UI showing real-time updates.
  - Implemented transparency abilities to comply with DSA (collapsible breakdown panels, math display).
  - Implemented both MAUT and hard constraint filtering modes.
- **Next steps:**
  - **Fix MAUT math bugs:** Resolve crashes on zero slider weights and correct percentage overflow issues.
  - **Hybrid mode:** Idea: Combine both modes (filter out expensive hotels, then rank the rest using MAUT).
  - **Expand mock database:** Add more hotels to the mock database.

Thank you for your attention!

Questions?

References I

[1] Joan Borras, Antonio Moreno und Aida Valls. „Intelligent tourism recommender systems: A survey“. In: *Expert Systems with Applications* 41.16 (2014), S. 7370–7389. ISSN: 0957-4174. DOI: https://doi.org/10.1016/j.eswa.2014.06.007. URL: https://www.sciencedirect.com/science/article/pii/S0957417414003431.

[2] European Parliament and Council of the European Union. *Regulation (EU) 2022/2065 of the European Parliament and of the Council of 19 October 2022 on a Single Market For Digital Services and amending Directive 2000/31/EC (Digital Services Act)*. 2022. URL: http://data.europa.eu/eli/reg/2022/2065/oj.

[3] Mohamed Elyes Ben Haj Kbaier, Hela Masri und Saoussen Krichen. „A Personalized Hybrid Tourism Recommender System“. In: *2017 IEEE/ACS 14th International Conference on Computer Systems and Applications (AICCSA)*. 2017, S. 244–250. DOI: 10.1109/AICCSA.2017.12.

[4] Joy Lal Sarkar u. a. „Tourism recommendation system: a survey and future research directions“. In: *Multimedia Tools and Applications* 82.6 (März 2023), S. 8983–9027. ISSN: 1573-7721. DOI: 10.1007/s11042-022-12167-w. URL: https://doi.org/10.1007/s11042-022-12167-w.

[5] Christian Schmitt, Dietmar Dengler, Mathias Bauer u. a. „The maut-machine: an adaptive recommender system“. In: *Proceedings of the ABIS Workshop, Hannover, Germany*. 2002.